ASSIGNMENT NO:16.2

Name:mdshahid

Ht.no:2303A51421 bt.no:21

Task Descrption-1: (Design Database Schema for Student Management System)

Instructions:

• Using AI tools, generate CREATE TABLE statements for a Student Management Database.

• Tables: Students, Courses, Enrollments.

• Include primary keys, foreign keys, and suitable data types.

Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Code & Output screenshot:

```sql
--Student Management Database.
--• Tables: Students, Courses, Enrollments.
--• Include primary keys, foreign keys, and suitable data types.
--Ensure the schema follows normalization rules (1NF, 2NF,3NF).
-- CREATE TABLE statement for Students
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    date_of_birth DATE NOT NULL
);
-- CREATE TABLE statement for Courses
CREATE TABLE Courses (

    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
-- CREATE TABLE statement for Enrollments
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
-- The above schema follows normalization rules:
-- 1NF: Each table has a primary key and atomic values.
-- 2NF: All non-key attributes are fully functional dependent on the
-- 3NF: There are no transitive dependencies; all attributes depend
-- key.
```

```sql
SQL ▾  1 / 1  1 - 0 of 0
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
date_of_birth DATE NOT NULL
);

SQL ▾  1 / 1  1 - 0 of 0
CREATE TABLE Courses (

course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
course_code TEXT UNIQUE NOT NULL,
credits INTEGER NOT NULL
);

SQL ▾  1 / 1  1 - 0 of 0
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
enrollment_date DATE NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Task Description- 2: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Code & Output screenshot :

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);


Select * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

SQL ▾ 1 / 1 1 - 4 of 4

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

SQL ▾ 1 / 1 1 - 3 of 3

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

SQL ▾ 1 / 1 1 - 5 of 5

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

```
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;

SELECT * FROM Courses;
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;



SELECT * FROM Courses;
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

SQL ▾ 1 / 1 1 - 0 of 0

```
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
```

SQL ▾ 1 / 1 1 - 0 of 0

```
DELETE FROM Enrollments
WHERE enrollment_id = 5;
```

SQL ▾ 1 / 1 1 - 4 of 4

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

SQL ▾ 1 / 1 1 - 3 of 3

student_id	name	email
1	Arjun	arjun_new@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

SQL ▾ 1 / 1 1 - 4 of 4

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Code &output screenshot:

Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

• Count number of students enrolled per course

• Display courses with more than one student enrolled

Analyze the correctness of results.

Code &output screenshot:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

SQL ▾ ❮ 1 / 1 ❯ 1 - 3 of 3

course_name	student_count
Data Structures	1
Database Systems	2
Operating Systems	1

SQL ▾ ❮ 1 / 1 ❯ 1 - 1 of 1

course_name
Database Systems

Task Description- 5: (Query Optimization)

Instructions:

• Use AI tools to analyze query performance.

• Rewrite inefficient sub queries using JOINs and test both queries for correctness

```
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
SELECT s.name, c.course_name
```

SQL ▾ ❮ 1 / 1 ❯ 1

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com